Systém pro správu skladu elektronických součástek

WMS – Elektronické součástky

Projektová dokumentace

Autoři: Tomáš Simandl
Tomáš Szetei
Lukáš Kvapil

Datum: 5. dubna 2026

Dokumentace systému pro skladovou evidenci elektronických součástek

Obsah

1 Úvod **2**

1.1 Motivace . . . 2

1.2 Cíle projektu . . . 2

1.3 Rozsah dokumentu . . . 3

1.4 Použité nástroje a technologie . . . 3

2 Analýza požadavků **4**

2.1 Funkční požadavky . . . 4

2.1.1 Správa součástek . . . 4

2.1.2 Skladová místa . . . 4

2.1.3 Příjem a výdej . . . 4

2.1.4 Objednávky a dodavatelé . . . 5

2.1.5 Inventura . . . 5

2.1.6 Reporty . . . 5

2.1.7 Správa uživatelů . . . 5

2.2 Nefunkční požadavky . . . 6

2.3 Případy užití – přehled . . . 6

3 Datový model **8**

3.1 Formulace problému . . . 8

3.2 Datový slovník . . . 8

3.3 Diagram tříd . . . 10

3.4 Stavový diagram . . . 12

3.5 Activity diagram – Vytvoření pohybu . . . 14

3.6 Sekvenční diagram – Vytvoření pohybu . . . 15

3.7 Use Case . . . 16

4 Popis procesů **17**

4.1 Proces příjmu součástek . . . 17

4.1.1 Popis kroků příjmu . . . 18

4.2 Proces výdeje součástek . . . 19

4.3 Proces inventury 20
4.3.1 Fáze inventurního procesu 21
4.4 Stav objednávky – stavový diagram 22

5 Architektura systému 23
5.1 Přehled architektury 23
5.2 REST API – přehled endpointů 25
5.3 Bezpečnostní model 25

6 Formální specifikace dotazů 27
6.1 Definice notace 27
6.2 Příklady dotazů nad modelem WMS 28
6.2.1 Součástky pod minimálním stavem 28
6.2.2 Celková hodnota skladu 28
6.2.3 Katalog součástek dodavatele 28
6.2.4 Pohyby výdeje na projekt 28
6.2.5 Složité dotazy – inventurní rozdíly 28

7 Závěr 30
7.1 Shrnutí 30
7.2 Další rozvoj 30
7.3 Diagram tříd – rozšiřitelnost 31

Kapitola 1

Analýza požadavků

1.1 Funkční požadavky

1.1.1 Správa součástek

FR-01 Systém umožní evidovat součástku s atributy: název, typ, výrobce, hodnota, tolerance, pouzdro (package), popis, datasheet URL.

FR-02 Systém podporuje kategorie součástek (rezistory, kondenzátory, tranzistory, diody, IC, konektory, ...).

FR-03 Každá součástka může mít přiřazený čárový kód (EAN/QR).

FR-04 Uživatel může vyhledávat součástky dle libovolné kombinace parametrů.

1.1.2 Skladová místa

FR-05 Systém spravuje hierarchii skladových míst: *Sklad* → *Sekce* → *Regál* → *Přihrádka*.

FR-06 Jedno skladové místo může obsahovat více druhů součástek; jedna součástka může být na více místech.

1.1.3 Příjem a výdej

FR-07 Příjem součástek ze skladu generuje pohyb typu `PŘÍJEM` a aktualizuje zásoby.

FR-08 Výdej součástek na projekt generuje pohyb typu `VÝDEJ` a sníží zásoby.

FR-09 Přeskladnění přesouvá zásoby mezi místy bez změny celkového množství.

FR-10 Systém hlídá minimální zásoby a upozorní na podkročení limitu.

1.1.4 Objednávky a dodavatelé

FR-11 Systém eviduje dodavatele s kontaktními údaji.

FR-12 Uživatel může vytvořit nákupní objednávku u dodavatele.

FR-13 Objednávka prochází stavy: `NÁVRH` → `ODESLÁNO` → `POTVRZENO` → `PŘIJATO` → `STORNOVÁNO`.

1.1.5 Inventura

FR-14 Systém umožní zahájit inventurní cyklus, během nějž uživatel ověřuje fyzické zásoby.

FR-15 Systém generuje inventurní list (PDF/CSV) se seznamem součástek a očekávaných množství.

FR-16 Po ukončení inventury systém provede korekci zásob.

1.1.6 Reporty

FR-17 Přehled aktuálních zásob s filtrací dle kategorie/umístění.

FR-18 Historie pohybů za zvolené časové období.

FR-19 Součástky pod minimálním limitem zásob.

FR-20 Ocenění skladu (celková hodnota zásob).

1.1.7 Správa uživatelů

FR-21 Systém spravuje uživatele s rolemi: `ADMIN`, `SKLADNÍK`, `ČTENÁŘ`.

FR-22 Každá akce je logována s časovým razítkem a uživatelem.

1.2 Nefunkční požadavky

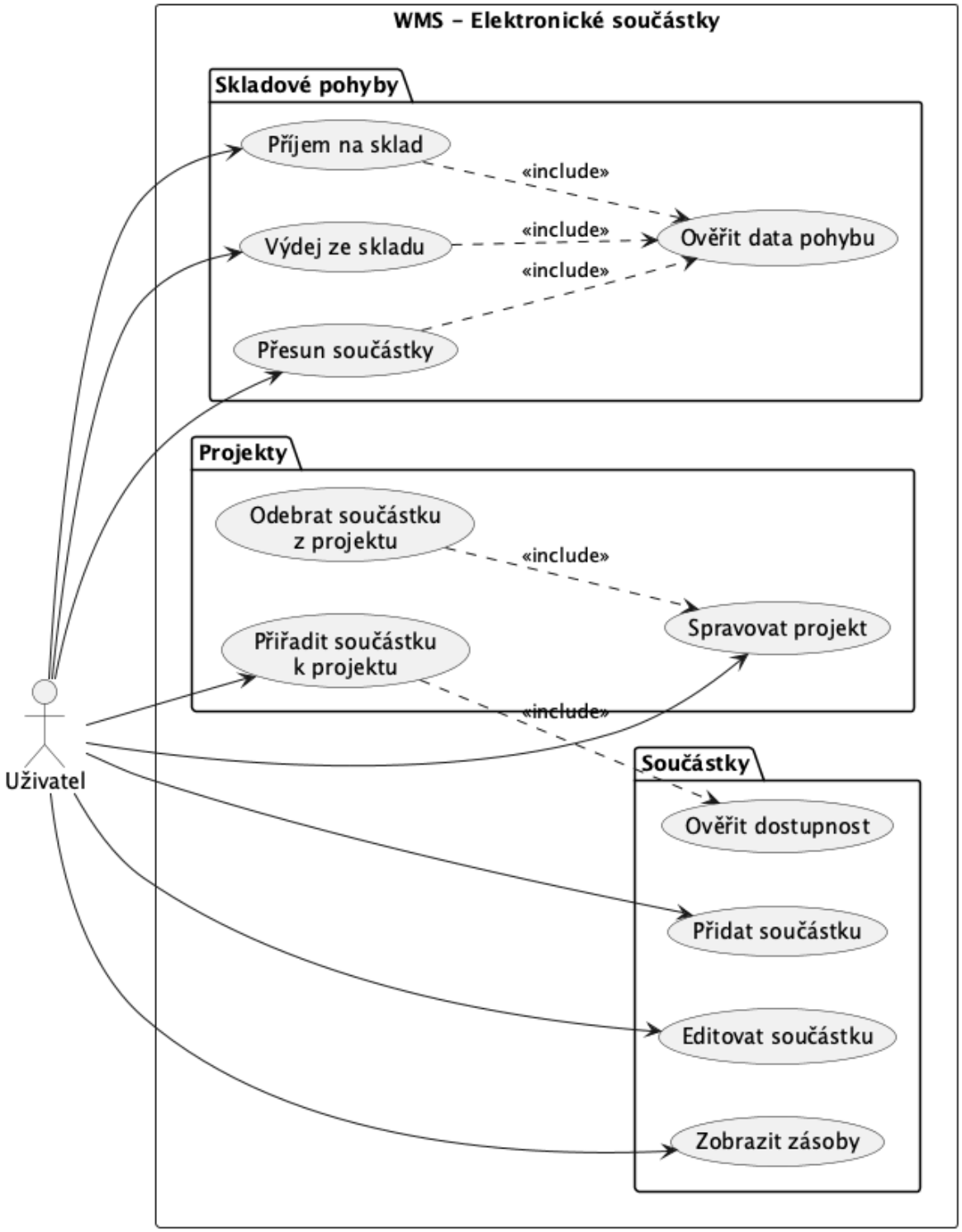
Tabulka 1.1: Nefunkční požadavky systému

ID	Kategorie	Požadavek
NFR-01	Výkon	Vyhledávání vrátí výsledky do 500 ms pro zásoby do 100 000 položek.
NFR-02	Dostupnost	Systém musí být dostupný 99,5 % času (plánovaná údržba vyjmuta).
NFR-03	Bezpečnost	Hesla uložena jako bcrypt hash; komunikace pouze přes HTTPS.
NFR-04	Škálovatelnost	Architektura umožní horizontální škálování backendu.
NFR-05	Přenositelnost	Nasazení pomocí Docker Compose; nezávislé na OS.
NFR-06	Použitelnost	Webové rozhraní přístupné bez speciálního SW v moderním prohlížeči.
NFR-07	Lokalizace	Primární jazyk čeština; systém připraven na vícejazyčnost (i18n).
NFR-08	Auditovatelnost	Každá změna stavu skladu musí být dohledatelná v auditním logu.

1.3 Případy užití – přehled

Hlavní aktér systému je **Uživatel**, který odpovídá třídě `Uzivatel` v datovém modelu. Uživatel provádí veškeré operace se systémem – vytváří skladové pohyby, spravuje součástky i projekty.

Podrobný Use Case diagram je zobrazen v Figure 2.1.



Obrázek 1.1: Use Case diagram systému WMS – elektronické součástky

Kapitola 2

Datový model

2.1 Formulace problému

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt. Návrh systému je znázorněn pomocí UML class diagramu, který popisuje jednotlivé třídy, jejich atributy a vztahy mezi nimi.

2.2 Datový slovník

Entity tříd

- **Soucastka**: Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě. Atribut `rezervovaneMnozstvi` sleduje, kolik kusů je aktuálně blokováno pro projekty – součástka tak může být *Dostupna* i s částečnou rezervací zároveň. Součástka může nabývat stavů: *Dostupna* (k dispozici, případně s rezervovanou částí), *Vydana* (fyzicky vydána ze skladu) nebo *Odepsana* (trvale vyřazena z evidence).

- **Projekt**: Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.

- **SkladovyPohyb**: Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun.

- **PrijemPohyb**: Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele (dědí ze **SkladovyPohyb**).

- **VydejPohyb**: Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu (dědí ze **SkladovyPohyb**). Atribut `duvod` popisuje účel výdeje, např. *použití v projektu*, *reklamace*, *interní spotřeba* nebo *oprava*.
- **PresunutiPohyb**: Konkrétní pohyb typu přesun – přemístnění součástek mezi lokáními ve skladu (dědí ze **SkladovyPohyb**).
- **Sklad**: Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.
- **Lokace**: Konkrétní místo v skladu identifikované označením (např. A-01-03).
- **Uzivatel**: Osoba spravující součástky, pohyby, projekty a sklady v systému.

Atributy

Tabulka 2.1: Přehled atributů tříd

Entita	**Atributy**
Soucastka	id, nazev, mnozstvi, rezervovaneMnozstvi, stav (Dostupna | Vydana | Odepsana)
Projekt	id, nazev
SkladovyPohyb	id, datum, stav, typ (prijem | vydej | presun), mnozstvi
PrijemPohyb	dodavatel – vlastní; id, datum, stav, typ – zděděno
VydejPohyb	duvod – vlastní; id, datum, stav, typ – zděděno
PresunutiPohyb	zdrojLokace, cilLokace – vlastní; id, datum, stav, typ – zděděno
Sklad	id, nazev
Lokace	id, oznaceni
Uzivatel	id, jmeno, email

2.3 Diagram tříd

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída `Uzivatel` reprezentuje uživatelské roli v systému. Vytvoření pohybu provádí voláním `SkladovyPohyb.vytvoritPohyb()` a přiřazení součástky k projektu voláním `Projekt.priradSouca`

Diagram využívá **generalizaci**: abstraktní třída `SkladovyPohyb` je základem pro tři konkrétní podtřídy `PrijemPohyb` (naskladnění), `VydejPohyb` (vyskladnění) a `PresunutiPohyb` (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

Třída `SkladovyPohyb` deklaruje abstraktní metodu `vytvoritPohyb()`, kterou každá podtřída implementuje po svém:

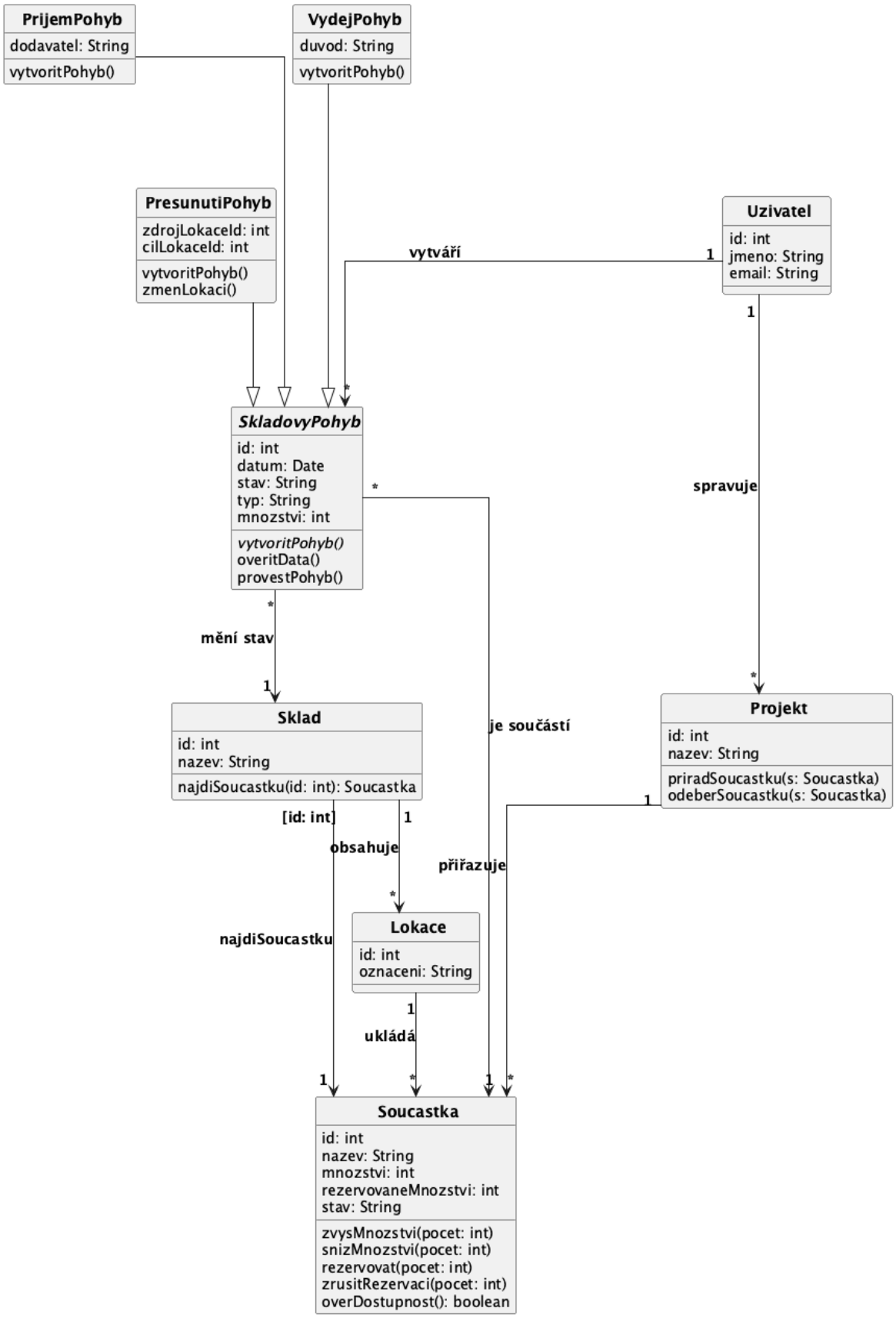
- `PrijemPohyb.vytvoritPohyb()` – vytvoří příjem a nastaví atribut `dodavatel`.
- `VydejPohyb.vytvoritPohyb()` – vytvoří výdej a nastaví atribut `duvod`.
- `PresunutiPohyb.vytvoritPohyb()` – vytvoří přesun a nastaví `zdrojLokaceId` a `cilLokaceId`.

Třída `SkladovyPohyb` zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- `PrijemPohyb` – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- `VydejPohyb` – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- `PresunutiPohyb` – přemístnění součástek mezi lokácemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba `je součástí`, multiplicita $* \rightarrow 1$) a mění stav jednoho skladu (vazba `mění stav`, multiplicita $* \rightarrow 1$). Jeden uživatel může vytvořit libovolný počet pohybů (vazba `vytváří`, multiplicita $1 \rightarrow *$). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou `Lokace` $\rightarrow$ `Soucastka`.

Vazba mezi `Sklad` a `Soucastka` je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru `id: int`.



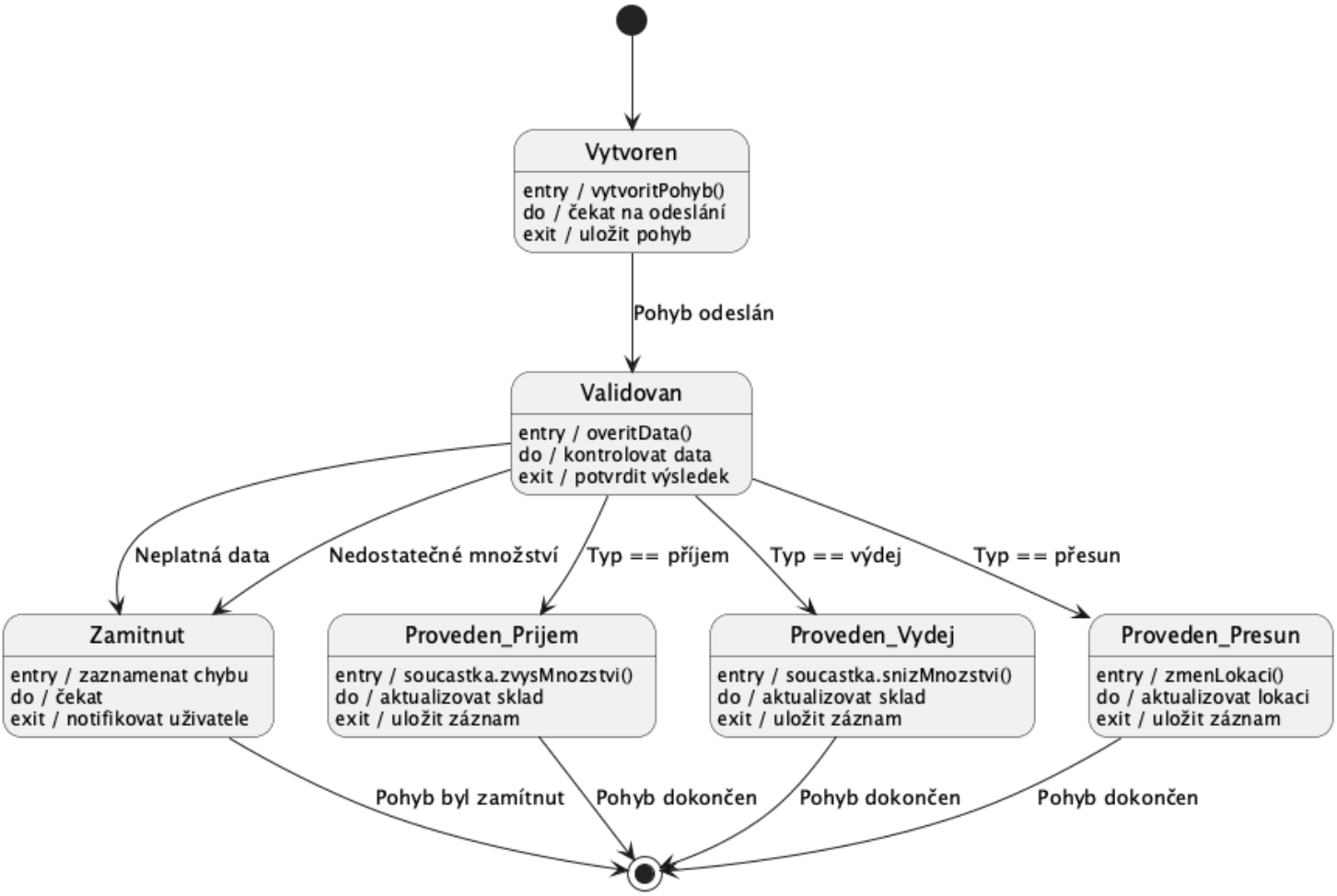
Obrázek 2.1: Class diagram WMS

2.4 Stavový diagram

Stavový diagram třídy `SkladovyPohyb`

Třída `SkladovyPohyb` reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- `vytvoritPohyb()` – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- `overitData()` – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- `provestPohyb()` – fyzicky provede pohyb: u příjmu zavolá `soucastka.zvysMnozstvi()`, u výdeje `soucastka.snizMnozstvi()` a u přesunu `zmenLokaci()`.

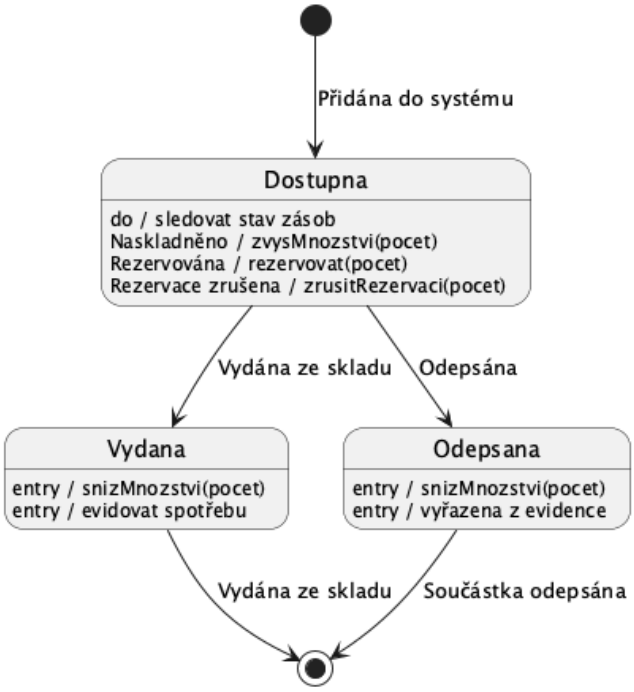


Obrázek 2.2: Stavový diagram třídy SkladovyPohyb

Stavový diagram třídy `Soucastka`

Diagram zachycuje, jak se mění stav součástky v průběhu skladových operací. Součástka je přidána do systému a okamžitě vstupuje do stavu *Dostupna* s nulovým množstvím. Stav *Dostupna* je hlavní pracovní stav součástky – může mít `mnozstvi = 0` (čeká na naskladnění) i nenulové množství, přičemž část může být blokována atributem `rezervovaneMnozstvi`. Metoda `overDostupnost()` ověří, zda je volné množství (`mnozstvi - rezervovaneMnozstvi > 0`). Stav součástky je řízen těmito metodami:

- `zvysMnozstvi(pocet)` – zavolá se při naskladnění; zvýší atribut `mnozstvi`, součástka zůstává ve stavu *Dostupna*.
- `rezervovat(pocet)` – zablokuje část kusů pro projekt; zvýší `rezervovaneMnozstvi`, stav se nemění.
- `zrusitRezervaci(pocet)` – uvolní rezervaci; sníží `rezervovaneMnozstvi`, stav se nemění.
- `snizMnozstvi(pocet)` – zavolá se při výdeji ze skladu (*Vydana*) nebo odpisu (*Odepsana*); oba jsou terminální stavy.

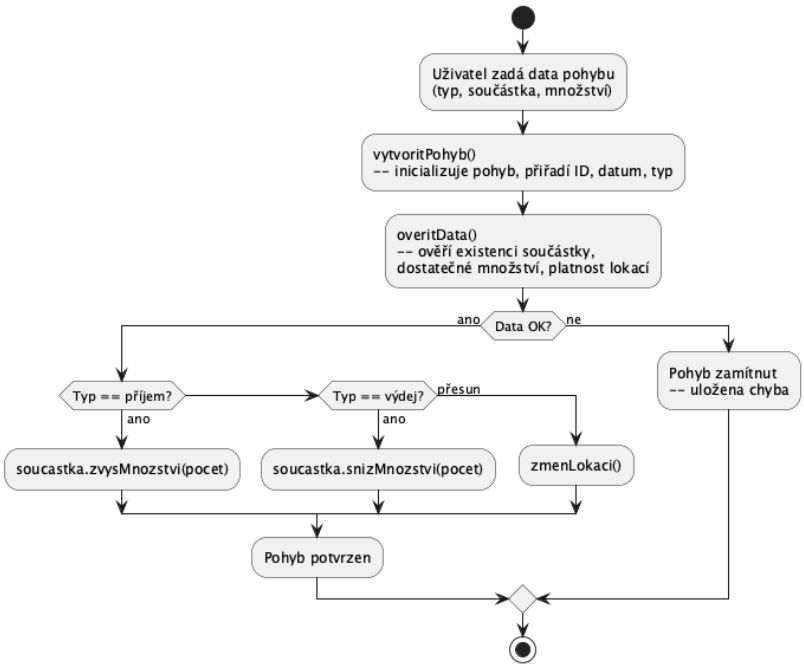


Obrázek 2.3: Stavový diagram třídy Soucastka

2.5 Activity diagram – Vytvoření pohybu

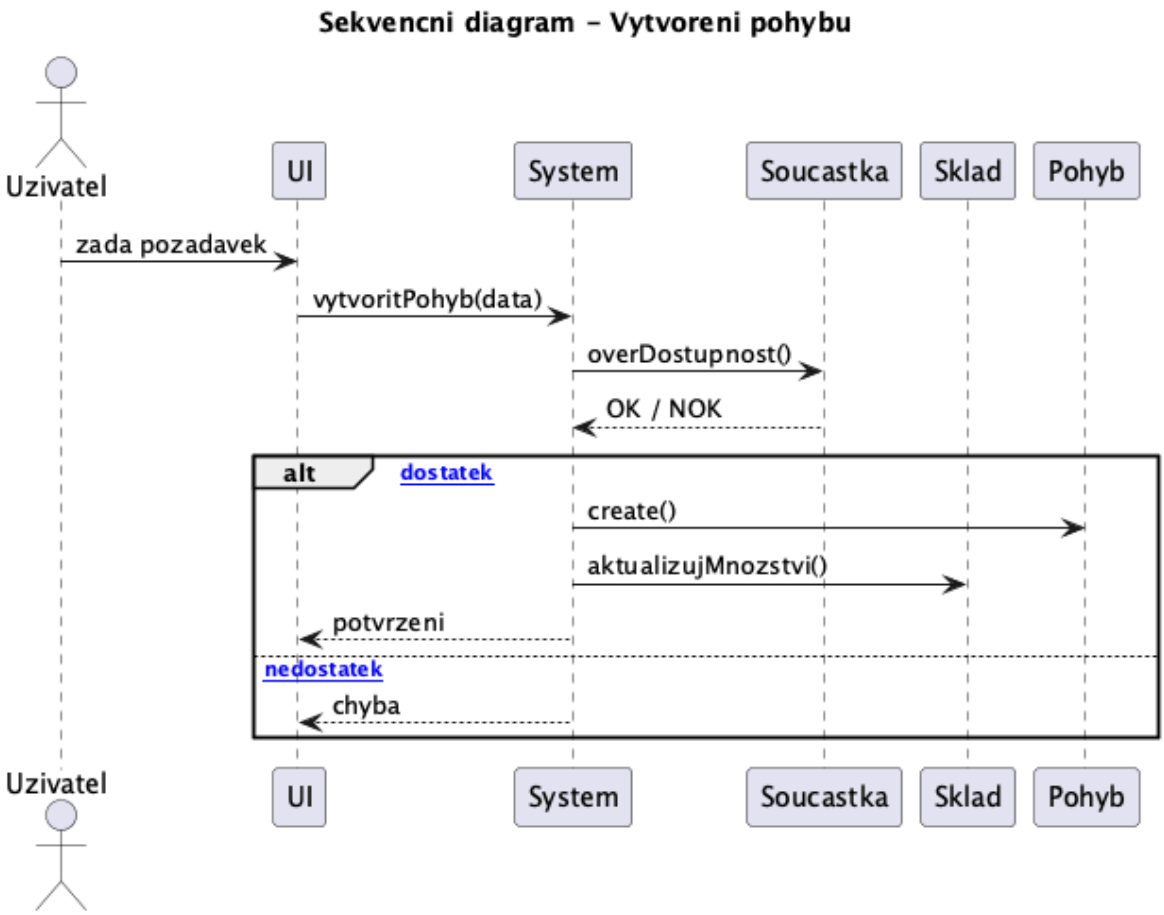
Diagram zachycuje průběh vytvoření skladového pohybu v systému WMS. Pohyb prochází třemi fázemi řízenými metodami třídy `SkladovyPohyb`:

- `vytvoritPohyb()` – uživatel zadá typ pohybu, součástku a množství; metoda inicializuje pohyb, přiřadí ID a datum vzniku.
- `overitData()` – ověří platnost zadaných dat: existenci součástky, dostatečné množství na skladě a platnost lokací u přesunu.
- `provestPohyb()` – fyzicky provede pohyb podle typu: příjem zavolá `zvysMnozstvi(pocet)`, výdej `snizMnozstvi(pocet)`, přesun `zmenLokaci()`. Pokud data nejsou platná, pohyb je zamítnut a uložena chyba.



Obrázek 2.4: Activity diagram – Vytvoření pohybu

2.6 Sekvenční diagram – Vytvoření pohybu



Obrázek 2.5: Sekvenční diagram – Vytvoření pohybu

2.7 Use Case

Use Case diagram systému je zobrazen v Figure 2.1 (kapitola Analýza požadavků). Z pohledu datového modelu jsou klíčové tyto scénáře, které přímo odpovídají metodám tříd `SkladovyPohyb` a `Projekt`:

Scénář: Vytvoření pohybu

1. Uživatel zvolí typ pohybu – příjem (`PrijemPohyb`), výdej (`VydejPohyb`) nebo přesun (`PresunutiPohyb`).
2. Zadá součástku a množství.
3. Systém volá `overitData()` – ověří existenci součástky, dostatečné množství a platnost lokací.
4. Pokud data OK → systém volá `provestPohyb()` a aktualizuje zásoby.
5. Pohyb je potvrzen; v případě chyby je zamítnut.

Scénář: Přiřazení součástky k projektu

1. Uživatel vybere projekt a zadá součástku s množstvím.
2. Systém volá `overDostupnost()` – ověří, zda `mnozstvi - rezervovaneMnozstvi > 0`.
3. Pokud je dostatek volných kusů → systém volá `Projekt.priradSoucastku()`.
4. Součástka je přiřazena k projektu; zásoby jsou aktualizovány.